Master: Bases de Données Fall, 2026

TP 2 (universityDB-data)

1 Setting up PostgreSQL

1. USING USPN LABS DATABASE

- Login to `linux` with your user account.
- Type the following command to start the `postgreSQL prompt`:

  ```
  psql -h postgres -U m1info_xx
  ```

  where `xx` is your account number between 1 and 90 $(1, 2, \ldots, 9, 10, \ldots, 90)$. The password is `M1INFO_xx`, with the same value of `xx` as your account number.
- You can see the list of tables present by typing :

  ```
  \d
  ```
- Download the following file in the same directory from which you ran the `psql` command:

  universityDB-schema.sql.

  This file contains the `SQL` code to create tables (I have given this code at the end of this subject).
- To read all the commands in the above file with `psql`, type :

  ```
  \i universityDB-schema.sql
  ```

  If this works, then on the left you can see the list of all the tables created.
- Once your tables are created, you can enter the data from the file universityDB-data.sql and run it. This file contains the `SQL` code to add data to our university database tables.

  ```
  \i universityDB-data.sql
  ```
- Now you can run whatever queries you want, for example :

  ```
  SELECT * FROM teacher;
  ```

Note: if you want to delete all the tables (*do this only if you did not create tables correctly*), you can run the following code.

```
drop table prereq;
drop table time_slot;
drop table advisor;
drop table takes;
drop table student;
drop table teaches;
drop table section;
drop table teacher;
drop table course;
drop table department;
drop table classroom;
```

2 Exercises

Now write the SQL code, and then run to verify, the following queries. The table after each question gives the result that you should get.

a.) Find the names and ids of students who have taken only one class.

Trouvez les noms et identifiants (ids) des étudiants qui n'ont suivi qu'un seul cours.

id	*name*
76653	Aoi
55739	Sanchez
19991	Brandt
44553	Peltier
23121	Chavez

Solution:

```
SELECT student.id, student.name 'abc'
FROM takes, student
WHERE student.id = takes.id
GROUP BY student.id, student.name
HAVING count(*) = 1;
```

b.) Find the name and budget of all departments with the maximum budget.

Trouvez le nom et le budget de tous les départements ayant le budget maximum.

dept_name	*budget*
Finance	120000.00

Solution:

```
SELECT dept_name, budget
FROM department
WHERE budget in
  (SELECT max(budget)
     FROM department);
```

c.) Find the lowest, across all departments, of the per-department maximum salary.

Trouvez, parmi tous les départements, le minimum du salaire maximum par département.

min
40000.00

Solution:

```
SELECT min(maxsalary)
FROM (SELECT dept_name, max(salary) as maxsalary
      FROM teacher
      GROUP BY dept_name) as mytable;
```

d.) Find the names and salaries of all faculty that earn more than the average salary.

Trouvez les noms et salaires de tous les enseignants qui gagnent plus que le salaire moyen.

name	*salary*
Wu	90000.00
Einstein	95000.00
Gold	87000.00
Katz	75000.00
Singh	80000.00
Brandt	92000.00
Kim	80000.00

Solution:

```
SELECT A.name,  A.salary
FROM teacher as A
WHERE A.salary >  (SELECT avg(B.salary)
       FROM teacher as B);
```

e.) Find the name of the student who has taken the most classes.

Trouvez le nom de l'étudiant qui a suivi le plus de cours.

name
Shankar

Solution:

```
SELECT student.name
FROM student, takes
WHERE student.id = takes.id
GROUP BY student.name
HAVING count(*) =
  (
  SELECT max(mynum)
  FROM (
    SELECT count(*) AS mynum
    FROM takes
    GROUP BY id
    ) AS mytable
  );
```

f.) For each department, find the maximum salary, and then return the minimum of all these maximum values. Return both the `dept_name` as well as the corresponding salary.

Pour chaque département, trouvez le salaire maximum, puis retournez le minimum de toutes ces valeurs maximales. Retournez à la fois le `dept_name` ainsi que le salaire correspondant.

dept_name	*min*
Music	40000.00

Solution:

```
SELECT A.dept_name, A.salary
FROM teacher AS A
WHERE A.salary =
  (SELECT min(s)
   FROM (SELECT max(salary) AS s
         FROM teacher
         GROUP BY dept_name) AS T);
```

g.) For each teacher, list all the student names of those students who have taken at least 2 classes from that teacher. **You must write this query using HAVING.**

Pour chaque enseignant, listez les noms de tous les étudiants qui ont suivi au moins 2 cours avec cet enseignant. **Vous devez écrire cette requête en utilisant HAVING.**

teachername	*studentname*	*totalcount*
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Srinivasan	Zhang	2

Solution:

```
SELECT teacher.name, student.name, count(*)
FROM teacher, student, takes, teaches
WHERE teacher.id = teaches.id and student.id = takes.id and (
   takes.course_id, takes.sec_id, takes.semester, takes.year)
   = (teaches.course_id, teaches.sec_id, teaches.semester,
   teaches.year)
GROUP BY teacher.name, student.name
HAVING count(*) >= 2;
```

h.) For each teacher, list all the student names of those students who have taken at least 2 classes from that teacher. **You must write this query without HAVING**.

Pour chaque enseignant, listez les noms de tous les étudiants qui ont suivi au moins 2 cours avec cet enseignant. **Vous devez écrire cette requête sans HAVING**.

teachername	*studentname*	*totalcount*
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Srinivasan	Zhang	2

Solution:

```
SELECT T.teachername, T.studentname, T.totalcount
FROM   (SELECT teacher.name as teachername, student.name as
   studentname, count(*) as totalcount
  FROM teacher, student, takes, teaches
  WHERE teacher.id = teaches.id and student.id = takes.id and
   (takes.course_id, takes.sec_id, takes.semester, takes.year)
    = (teaches.course_id, teaches.sec_id, teaches.semester,
   teaches.year)
  GROUP BY teacher.name, student.name) as T
WHERE T.totalcount >= 2
ORDER BY T.teachername;
```

i.) Find all departments where teachers' total salary is greater than the average of the total salary of all departments.

Trouvez tous les départements où le salaire total des enseignants est supérieur à la moyenne du salaire total de tous les départements.

dept_name
Comp. Sci.
Finance
Physics

```
SELECT DISTINCT T1.dept_name
FROM teacher as T1
WHERE
  (SELECT sum(salary)
   FROM teacher as T2
   WHERE T2.dept_name = T1.dept_name)
         >=
  (SELECT avg(s)
   FROM
    (SELECT sum(salary) as s
     FROM teacher as T3
     GROUP BY T3.dept_name) as T4
  );
```

j.) Find the names and salaries of the teachers among the 3 lowest salaries, sorted by increasing salary. **You must write this query using HAVING.**

Trouvez les noms et salaires des enseignants parmi les 3 salaires les plus bas, triés par salaire croissant. **Vous devez écrire cette requête en utilisant HAVING.**

name	*salary*
Mozart	40000.00
El Said	60000.00
Califieri	62000.00

```
SELECT T1.name, T1.salary
FROM teacher AS T1, teacher AS T2
WHERE T2.salary <= T1.salary
GROUP BY T1.id, T1.name
HAVING count(*) <= 3
ORDER BY T1.salary;
```

k.) Find the names and salaries of the teachers among the 3 lowest salaries, sorted by increasing salary. **You must write this query with nested subqueries.**

Trouvez les noms et salaires des enseignants parmi les 3 salaires les plus bas, triés par salaire croissant. **Vous devez écrire cette requête avec des sous-requêtes imbriquées.**

name	*salary*
Mozart	40000.00
El Said	60000.00
Califieri	62000.00

```
SELECT T1. name, T1.salary
FROM teacher as T1
WHERE 2 >= (
  SELECT COUNT (DISTINCT T2.salary)
  FROM teacher as T2
  WHERE T2.salary < T1.salary)
ORDER BY T1.salary ASC;
```

l.) Find courses that have sections taught in different buildings. Use **HAVING**.

Trouvez les cours qui ont des sections enseignées dans des bâtiments différents. Utilisez **HAVING**.

course_id	*title*	*building_count*
CS-319	Image Processing	2

```
SELECT c.course_id, c.title, count(DISTINCT s.building) as
   building_count
FROM course AS c, section AS s
WHERE c.course_id = s.course_id
GROUP BY c.course_id, c.title
HAVING count(DISTINCT s.building) > 1;
```

universityDB-schema.sql

```
create table classroom
  (building    varchar(15),
   room_number    varchar(7),
   capacity   numeric(4,0),
   primary key (building, room_number)
  );

create table department
  (dept_name    varchar(20),
   building   varchar(15),
   budget          numeric(12,2),
   primary key (dept_name)
  );

create table course
  (course_id    varchar(8),
   title      varchar(50),
   dept_name    varchar(20),
   credits    numeric(2,0),
   primary key (course_id),
   foreign key (dept_name) references department
  );

create table teacher
  (ID     varchar(5),
   name     varchar(20),
   dept_name    varchar(20),
   salary     numeric(8,2),
   primary key (ID),
   foreign key (dept_name) references department
  );

create table section
  (course_id    varchar(8),
       sec_id     varchar(8),
   semester   varchar(6)
    check (semester in ('Fall', 'Winter', 'Spring', 'Summer')),
   year     numeric(4,0),
   building   varchar(15),
   room_number    varchar(7),
   time_slot_id   varchar(4),
   primary key (course_id, sec_id, semester, year),
   foreign key (course_id) references course,
   foreign key (building, room_number) references classroom
  );

create table teaches
  (ID     varchar(5),
   course_id    varchar(8),
   sec_id     varchar(8),
   semester   varchar(6),
   year     numeric(4,0),
   primary key (ID, course_id, sec_id, semester, year),
   foreign key (course_id,sec_id, semester, year) references section,
   foreign key (ID) references teacher
  );

create table student
  (ID     varchar(5),
   name     varchar(20),
   dept_name    varchar(20),
   tot_cred   numeric(3,0),
   primary key (ID),
   foreign key (dept_name) references department
  );

create table takes
  (ID     varchar(5),
   course_id    varchar(8),
   sec_id     varchar(8),
   semester   varchar(6),
   year     numeric(4,0),
   grade          varchar(2),
   primary key (ID, course_id, sec_id, semester, year),
   foreign key (course_id,sec_id, semester, year) references section,
   foreign key (ID) references student
  );

create table advisor
  (s_ID     varchar(5),
   i_ID     varchar(5),
   primary key (s_ID),
   foreign key (i_ID) references teacher (ID),
   foreign key (s_ID) references student (ID)
  );

create table time_slot
  (time_slot_id   varchar(4),
   day      varchar(1),
   start_hr   numeric(2),
   start_min    numeric(2),
   end_hr     numeric(2),
   end_min    numeric(2),
   primary key (time_slot_id, day, start_hr, start_min)
  );

create table prereq
  (course_id    varchar(8),
   prereq_id    varchar(8),
   primary key (course_id, prereq_id),
   foreign key (course_id) references course,
   foreign key (prereq_id) references course
  );
```

Tables

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

(a) course

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

(b) teacher

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

(c) student

course_id	*sec_id*	*semester*	*year*	*building*	*rn*	*time_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

(d) section

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

(e) teaches

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

(f) takes

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

(g) department

building	*room_number*	*capacity*
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

(h) classroom

3 ONLY IF LINUX DOES NOT WORK

Here is how to use `PostgreSQL` online.

2. USING ONLINE DATABASE

- Go to the site

  `https://sqliteonline.com/`

- On the left, choose '`PostgreSQL`' and click on '`Click to connect`'.

- In the main window, create the database for the university by copy-pasting and running the code in file universityDB-schema.sql. This file contains the `SQL` code to create tables (I have given this code at the end of this subject).

  If this works, then on the left you can see the list of all the tables created.

- Once your tables are created, you can enter the data from the file universityDB-data.sql and run it. This file contains the `SQL` code to add data to our university database tables (I have given this code at the end of this subject).